



UNIVERSIDAD DE LOS LAGOS

GESTOR DE TAREAS

LABORATORIO 2

DEPARTAMENTO DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA CIVIL EN INFORMÁTICA
TALLER DE INGENIERÍA INFORMÁTICA
CAMPUS OSORNO, CHILE

Boris Carrasco
Cristian Malhue
Nelson Espinoza
Matias Guzman

3 de junio de 2026



5 años **UNIVERSIDAD ACREDITADA**
Septiembre de 2021 - Septiembre de 2026
Gestión Institucional - Docencia de Pregrado
Investigación - Vinculación con el Medio
AVANZADA

www.ulagos.cl

Índice general

1. Introduccion	1
1.1. Repositorio github	1
1.2. Estructura del proyecto	1
1.3. Configuración de Docker	2
1.3.1. Docker compose	2
1.3.2. Configuración del archivo docker-compose.yml	2
1.3.3. Dockerfile	3
2. Desarrollo	4
2.1. Construcción	4
2.2. Ejecución	5
2.3. Logs/Red	6
2.4. Prueba funcional	6
3. Conclusion	7
A. Anexos	8
A.1. Anexos del Trabajo	8
A.1.1. docker-compose.yml	8
A.1.2. Dockerfile	9
A.1.3. Link repositorio github	9

Índice de figuras

1.1. Estructura proyecto	2
2.1. docker compose build pt 1	4
2.2. docker compose build pt 2	5
2.3. docker compose up pt 1	5
2.4. docker compose up pt 2	5
2.5. docker compose ps	5
2.6. Logs de base de datos y red	6
2.7. Prueba funcional	6

Índice de tablas

Índice de algoritmos

A.1. Codigo docker-compose.yml	8
A.2. Codigo docker-compose.yml	9

Capítulo 1

Introduccion

En este informe de proyecto se mostraran las pruebas de construccion y despliege de un proyecto que consiste en una aplicación web para gestionar grupos y tareas, permitiendo crear, ver, editar y eliminar información de forma sencilla.

Para su ejecución se usó Docker, separando la aplicación y la base de datos en contenedores distintos. En el informe se muestran la estructura del proyecto, la configuración usada y las pruebas que demuestran su funcionamiento

1.1. Repositorio github

Repositorio del proyecto en GitHub

1.2. Estructura del proyecto

El proyecto se organiza en distintas carpetas y archivos, separando la aplicación web, la base de datos, los recursos visuales y los archivos de configuración de Docker. La estructura general es la siguiente:

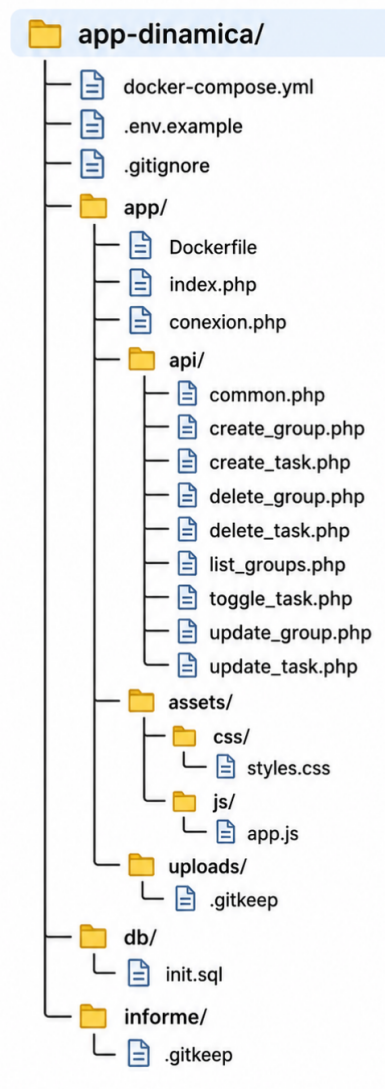


Figura 1.1: Estructura proyecto

1.3. Configuración de Docker

1.3.1. Docker compose

1.3.2. Configuración del archivo `docker-compose.yml`

El archivo `docker-compose.yml` permite levantar y conectar los servicios principales del proyecto mediante Docker Compose. En este caso, se definen dos contenedores: uno para la aplicación web y otro para la base de datos MySQL.

- **Servicio aplicacion:** ejecuta la aplicación web en un contenedor con Apache y PHP. Se construye desde la carpeta `./app` utilizando su respectivo `Dockerfile`.
- **Puerto de acceso:** el puerto interno 80 del contenedor se expone mediante la variable `APP_PORT`, usando por defecto el puerto 8080. Por esto, la aplicación puede abrirse desde `localhost:8080`.
- **Conexión con la base de datos:** la aplicación utiliza variables de entorno para conectarse a MySQL. El host definido es `base_datos`, que corresponde al nombre del servicio de base de datos dentro de Docker Compose.
- **Volumen de la aplicación:** se monta la carpeta local `./app` en `/var/www/html`, permitiendo reflejar los cambios del código dentro del contenedor.
- **Servicio base_datos:** utiliza la imagen oficial `mysql:8.0`. Este contenedor crea la base de datos, el usuario y las contraseñas necesarias para el funcionamiento del sistema.
- **Persistencia de datos:** se utiliza el volumen `datos_mysql` en la ruta `/var/lib/mysql`, para mantener la información aunque el contenedor sea detenido o recreado.
- **Inicialización de la base de datos:** se monta el archivo `init.sql`, encargado de crear automáticamente la estructura inicial de la base de datos.
- **Red interna:** ambos servicios se conectan mediante la red `red_aplicacion`, lo que permite la comunicación entre la aplicación web y MySQL.

1.3.3. Dockerfile

El archivo `Dockerfile` permite construir la imagen del contenedor donde se ejecuta la aplicación web. Para esto, utiliza una imagen base con PHP y Apache.

- **Imagen base:** se utiliza `php:8.2-apache`, que incluye PHP junto con el servidor Apache.
- **Extensiones necesarias:** se instalan `pdo` y `pdo_mysql`, necesarias para conectar la aplicación con la base de datos MySQL.
- **Directorio de trabajo:** se define la ruta `/var/www/html`, que corresponde al lugar donde Apache ejecuta los archivos del proyecto.
- **Carpeta de archivos:** se crea la carpeta `uploads`, destinada a almacenar archivos dentro de la aplicación.
- **Permisos:** se configuran permisos para que el servidor web pueda acceder correctamente a la carpeta `uploads`.
- **Puerto expuesto:** se expone el puerto 80, utilizado por Apache para mostrar la aplicación en el navegador.

Capítulo 2

Desarrollo

2.1. Construcción

Se ejecuta `docker compose build` para construir la imagen de la aplicación web. En este paso, Docker descarga la imagen base de PHP con Apache y prepara el contenedor según las instrucciones del Dockerfile.

```

root@192.168.1.102: ~$ docker compose build
[+] Building 7.6s (5/9)
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 522B                                          0.0s
=> [internal] load build definition from Dockerfile                    0.1s
=> => transferring dockerfile: 265B                                     0.0s
=> [internal] load metadata for docker.io/library/php:8.2-apache      2.5s
=> [internal] load .dockerignore                                       0.0s
=> => transferring context: 2B                                          0.0s
[1/4] FROM docker.io/library/php:8.2-apache@sha256:affc043fbd9acaa9a6394a71d162726fc0a6e4bea0400a3b94f925b6130858dd
=> resolve docker.io/library/php:8.2-apache@sha256:affc043fbd9acaa9a6394a71d162726fc0a6e4bea0400a3b94f925b6130858dd  4.7s
=> sha256:4f4fb700ef54461cf02571ae0db9a0dc1e0c0b5577484a6d75e68dc38e8acc1 32B / 32B
=> sha256:7596c9d2949eaf0ba880fc8fe74c060e95caae1f670ff2b0e6a9668923c011ca 892B / 892B
=> sha256:ffe6fb3b32696203fc5956a86e27974f859e091fad8d64099427e088d029e85f 249B / 249B
=> sha256:912c3b20bc776a00f64f500214e435800770a7c7d5635637c7086d0408608a 255B / 255B
=> sha256:1b41185227652eb7cc95ebaa8f530b3a1b98113192ebc81ee973cd05b96101ff 2.46kB / 2.46kB
=> sha256:e94c55ba5ab821149f8bc67d1f1a38892d2150bac0908a069cda8b29188 11.46MB / 11.46MB
=> sha256:e4885b58575c91f8adb1c29f0557959fb195a5cf63174e2430877af9840c5211 488B / 488B
=> sha256:72950e9a0ba87c0c7e2c0895309ac3e71eefc79339e2798eb43a5ca1632baef8 5.24MB / 12.33MB
=> sha256:bc3476edda7999ef33bfbb9f621ed871892ed96223ef0a0b4649f54f0fab09b 488B / 488B
=> sha256:d067e157fa180ba34216a6c0ca2b75ade07875001157fe42b0b0a07809a5 435B / 435B
=> sha256:f82bf7789d7e61ec9e65d8692b087b0f8a77eabbc524375c958ca663132a46 2.10MB / 4.23MB
=> sha256:5cd00f85f90e7677b1345e07c00c5e1dd24abc7847b2caf62356638e849f25b 226B / 226B
=> sha256:487e971560f0056dc9521874e757ab1114f0ea303f54e70f2a5bca1a6e53fc39 1.05MB / 117.84MB
=> sha256:34eb7071422ecd73d7df21a7467f66fcd3bb19f4dbc9328eaa86026a7e781f57 225B / 225B
=> sha256:5b4d6ff92fc4e14e911b7753c954fac965d48c40fe1075758d284148ccace970 0B / 29.78MB
=> [internal] load build context                                       0.3s
=> => transferring context: 112.04kB                                    0.0s

```

Figura 2.1: docker compose build pt 1

```

root@192.168.1.10322: ~# ssh root@192.168.1.71
Last login: Sat May 23 18:13:11 2026 from 192.168.1.71
root@debian:~# cd app-dinamica/
root@debian:~/app-dinamica# cp .env.example .env
root@debian:~/app-dinamica# docker compose build
[+] Building 79.2s (11/11) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 522B 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 265B 0.0s
=> [internal] load metadata for docker.io/library/php:8.2-apache 1.5s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
[1/4] FROM docker.io/library/php:8.2-apache@sha256:affc043fbd9acaa9a6394a71d162726fc0a6edbea0400a3b94f925b6130858dd 59.4s
=> => resolve docker.io/library/php:8.2-apache@sha256:affc043fbd9acaa9a6394a71d162726fc0a6edbea0400a3b94f925b6130858dd 0.0s
=> => sha256:487e971560f0056dc9521874e757ab1114f0ea303f54e70f2a5bca1ae53fc39 117.84MB / 117.84MB 49.4s
=> => extracting sha256:487e971560f0056dc9521874e757ab1114f0ea303f54e70f2a5bca1ae53fc39 7.3s
=> => extracting sha256:5cd80f85f90e7677b1345e07c60dc5e1da24abc7847b2ca62356638e849f25b 0.0s
=> => extracting sha256:f82b77804f0d61c0e65d8092b087b0a847eab0ca5237b3d385e6d63132246 0.3s
=> => extracting sha256:d867a15f7ac10dbce34216e5bca2b275ede97675081157c6e42b8dc87005e5 0.0s
=> => extracting sha256:bc3476adda7999ef33bfbbef9621ed871892ed9622ef0a8b4649f54f0fab69b 0.0s
=> => extracting sha256:72950e9a0ba87c0c7e2c0895309ac3e71eeefc79339e2798eb43a5ca1632baef8 0.2s
=> => extracting sha256:e4885b58575c91f8adb1c29f0557959fb195a5cf63174e2430877af9840c5211 0.0s
=> => extracting sha256:e694c35b9a5ab21149fc8bc67d1f1ae38892d2150bac090a069ceda0b29108 1.1s
=> => extracting sha256:1b411852f652ab7c950bae0f52003a1b90113192cc0ce973cd05b0a1aff 0.0s
=> => extracting sha256:9a2c3b28bcf76a00f0f150b921dec850840778b707d56366d765286d0408608e 0.0s
=> => extracting sha256:ffe0fb363269c203fc5956e86e27974f859e091fad8d64099427a088d029e05f 0.0s
=> => extracting sha256:7596c9d2949eaf0ba880fc8f674c069e95caae1fc70ff2b8a6a9668923c611a 0.0s
=> => extracting sha256:4f4fb700ef5461cfa02571ae0db9a0dc1e0c5577484a6d75e08dc38e8acc1 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 691B 0.0s
[2/4] RUN docker-php-ext-install pdo_pdo_mysql 16.7s
[3/4] COPY . /var/www/html/ 0.1s
[4/4] RUN mkdir -p /var/www/html/uploads && chown -R www-data:www-data /var/www/html && chmod -R 775 /var/www/html/uploads 0.3s
=> exporting to image 0.6s
=> => exporting layers 0.4s
=> => exporting manifest sha256:e4e654a0f2b7bd4899ec707ed129d010fe4116ae0606dc39a7f87368776761f 0.0s
=> => exporting config sha256:4d48e4b024cc563fcef4d90b0a85b86a6d363eaf66ddc72e2ab63039e1a925 0.0s
=> => exporting attestation manifest sha256:3b7500fd2279d093518a3f6b294240cd2719bcf0e1d44c0282c4eaf53ac5143 0.0s
=> => exporting manifest list sha256:f00130b26c3eaa94f15cf0f67b0e9981e21f6ff9b79bca531fb9c71d6f5772 0.0s
=> => naming to docker.io/library/app-dinamica:aplicacion:latest 0.0s
=> => unpacking to docker.io/library/app-dinamica:aplicacion:latest 0.1s
=> => resolving provenance for metadata file 0.0s
[+] build 1/1 79.3s
Image app-dinamica:aplicacion Built

```

Figura 2.2: docker compose build pt 2

2.2. Ejecucion

Se ejecuta docker compose up -d para iniciar los contenedores del proyecto en segundo plano. En este paso, Docker descarga la imagen de MySQL necesaria para la base de datos.

Se ejecuta docker ps para comprobar que los contenedores del proyecto están activos. En la salida se observa que la aplicación web y la base de datos MySQL se encuentran en ejecución, con sus puertos correspondientes habilitados.

```

root@debian:~/app-dinamica# docker compose up -d
[+] up 7/14
Image mysql:8.0 [ 0 0 0 0 ] Pulling 4.8s

```

Figura 2.3: docker compose up pt 1

```

root@debian:~/app-dinamica# docker compose up -d
[+] up 18/18
Image mysql:8.0 Pulled 739.9s
Network app-dinamica_red_aplicacion Created 0.1s
Volume app-dinamica_datos_mysql Created 0.0s
Container aplicacion_dinamica_bd Started 1.1s
Container aplicacion_dinamica_web Started 0.9s
root@debian:~/app-dinamica#

```

Figura 2.4: docker compose up pt 2

```

root@debian:~/app-dinamica# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                NAMES
8f0d461fef7b   app-dinamica:aplicacion             "docker-php-entrypoi..." 4 minutes ago  Up 4 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  aplicacion_dinamica_web
5df825fbaa70   mysql:8.0                           "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes (healthy)  0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp  aplicacion_dinamica_bd
root@debian:~/app-dinamica#

```

Figura 2.5: docker compose ps

2.3. Logs/Red

Los logs muestran que el contenedor de MySQL inició correctamente y está listo para recibir conexiones en el puerto 3306, confirmando que la base de datos funciona de manera adecuada.

```
aplicacion_dinamica_bd | 2026-05-23T22:41:37.112134Z | [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
aplicacion_dinamica_bd | 2026-05-23T22:41:37.112260Z | [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
aplicacion_dinamica_bd | 2026-05-23T22:41:37.116582Z | [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosi
ng a different directory.
aplicacion_dinamica_bd | 2026-05-23T22:41:37.139293Z | [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.46' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Communit
y Server - GPL.
```

Figura 2.6: Logs de base de datos y red

2.4. Prueba funcional

Luego de levantar los contenedores con Docker Compose, se ingresó a la aplicación desde el navegador

Con esta prueba se comprobó que la página cargaba correctamente y que el sistema estaba listo para ser utilizado.



Figura 2.7: Prueba funcional

Capítulo 3

Conclusion

En conclusion, se logró crear una aplicación web para gestionar grupos y tareas, permitiendo agregar, ver, editar y eliminar información. Con esto, se cumple el funcionamiento CRUD pedido en el laboratorio.

También se usó Docker para ordenar mejor el proyecto, separando la aplicación web y la base de datos MySQL en contenedores distintos. Gracias a Docker Compose, fue posible levantar todo el sistema de forma más simple.

Además, se trabajó con volúmenes para mantener los datos guardados y con una red interna para que los contenedores se comuniquen correctamente.

En resumen, el proyecto permitió aplicar de forma práctica el uso de aplicaciones web, bases de datos y Docker, logrando una solución funcional y bien organizada.

Anexos A

Anexos

A.1. Anexos del Trabajo

A.1.1. docker-compose.yml

Código A.1: Codigo docker-compose.yml

```
1 services:
2   aplicacion:
3     build: ./app
4     container_name: aplicacion_dinamica_web
5     ports:
6       - "${APP_PORT:-8080}:80"
7     environment:
8       DB_HOST: base_datos
9       DB_PORT: 3306
10      DB_NAME: ${MYSQL_DATABASE:-aplicacion_dinamica}
11      DB_USER: ${MYSQL_USER:-usuario_app}
12      DB_PASSWORD: ${MYSQL_PASSWORD:-clave_app}
13    volumes:
14      - ./app:/var/www/html
15    depends_on:
16      - base_datos
17    networks:
18      - red_aplicacion
19  base_datos:
20    image: mysql:8.0
21    container_name: aplicacion_dinamica_bd
22    restart: unless-stopped
23    environment:
24      MYSQL_DATABASE: ${MYSQL_DATABASE:-aplicacion_dinamica}
25      MYSQL_USER: ${MYSQL_USER:-usuario_app}
```

```

26     MYSQL_PASSWORD: ${MYSQL_PASSWORD:-clave_app}
27     MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:-clave_root}
28 ports:
29     - "${DB_PORT:-3306}:3306"
30 volumes:
31     - datos_mysql:/var/lib/mysql
32     - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
33 healthcheck:
34     test: ["CMD-SHELL", "mysqladmin ping -h localhost -uroot -p${MYSQL_ROOT_PASSWORD} --silent"]
35     interval: 10s
36     timeout: 5s
37     retries: 10
38     start_period: 20s
39 networks:
40     - red_aplicacion
41 volumes:
42     datos_mysql:
43 networks:
44     red_aplicacion:
45     driver: bridge

```

A.1.2. Dockerfile

Código A.2: Codigo docker-compose.yml

```

1 FROM php:8.2-apache
2 RUN docker-php-ext-install pdo pdo_mysql
3 WORKDIR /var/www/html
4 RUN mkdir -p /var/www/html/uploads \
5     && chown -R www-data:www-data /var/www/html \
6     && chmod -R 775 /var/www/html/uploads
7 EXPOSE 80

```

A.1.3. Link repositorio github

<https://github.com/kuka1409/taller-de-ingenieria>